

Relatório Final — Atividade de Extensão II

Ceres Diagnóstico — Sistema embarcado de detecção precoce de doenças no tomateiro · IFMT Cuiabá ·
Namem Rachid Jaudy Neto

1. Identificação do Projeto

1.1 Título

Ceres Diagnóstico — Sistema embarcado de detecção precoce de doenças no tomateiro, integrando visão computacional (TinyML) no ESP32-S3, backend em nuvem e aplicativo móvel.

1.2 Curso / Disciplina / Professor

- **Curso:** Engenharia da Computação — IFMT Cuiabá (Campus Cel. Octayde Jorge da Silva)
- **Disciplina:** Extensão II — 2026/1
- **Professor:** Tiago Lacerda

1.3 Integrantes

Projeto desenvolvido individualmente:

Nome	Funções desempenhadas
Namem Rachid Jaudy Neto	Product Owner, Scrum Master, Desenvolvedor Back-end (Django/API), Desenvolvedor Front-end/Mobile (Flutter), Engenheiro de IA (treino/quantização), Desenvolvedor de Firmware (ESP32-S3), Banco de Dados, UX/UI e QA

1.4 Local e Ano

Cuiabá — MT, IFMT, 2026.

2. Resumo Executivo

Problema. Pequenos produtores de tomate têm dificuldade de acesso a diagnóstico agrônômico rápido. Doenças como a requeima podem destruir uma lavoura inteira antes de o produtor identificar o problema, e a conectividade no campo é limitada.

Público-alvo. Pequenos horticultores de Sorriso-MT e região, produtores de tomate que trabalham diretamente no campo.

Produto. Um sistema em três camadas: (1) aplicativo móvel que diagnostica doenças da folha por foto, com IA rodando no próprio celular (offline) ou na nuvem; (2) nó IoT ESP32-S3 que monitora temperatura e umidade e roda o modelo embarcado; (3) backend em nuvem (Django REST no Railway) com histórico, mapa de ocorrências e enciclopédia.

Tecnologias. Django REST Framework, PostgreSQL, Flutter/Dart, TensorFlow/TFLite (MobileNetV2 INT8), ESP32-S3 (PlatformIO/Arduino), MQTT (Mosquitto/HiveMQ), Railway, GitHub.

Principais resultados. Modelo de **638 KB** classificando **10 categorias** (9 doenças + saudável); acurácia de **95,76%** (INT8, laboratório); latência de **692 ms** no ESP32-S3 e **~60 ms** on-device no celular; aplicativo completo com 5 telas publicado e API em produção. O projeto documentou honestamente o gap laboratório-campo (queda para 20–30% em imagens de campo real), fenômeno reconhecido na literatura.

3. Visão do Projeto

Problema identificado. Um tomateiro doente não “grita”: murcha em silêncio enquanto a doença avança. O diagnóstico depende de um agrônomo, inacessível ao pequeno produtor, e existem cerca de 10 doenças difíceis de distinguir a olho nu. A internet que permitiria o diagnóstico digital muitas vezes não chega à lavoura.

Público-alvo e comunidade beneficiada. Pequenos horticultores de Sorriso-MT e região — produtores que trabalham diretamente no campo, com pouco tempo ou dificuldade para digitar textos técnicos no celular.

Justificativa. Diferente de grupos de WhatsApp, buscas no Google ou chatbots agrícolas genéricos, o Ceres transforma recomendações de manejo (base Embrapa) em diagnósticos rápidos e acessíveis diretamente na lavoura, ajudando o produtor a reduzir perdas.

Objetivos.

- Treinar um modelo de IA leve o suficiente para rodar em microcontrolador e celular;
- Disponibilizar diagnóstico por foto funcionando offline;
- Monitorar variáveis ambientais da lavoura via sensores IoT;
- Entregar um aplicativo completo, publicado e utilizável em condições reais.

Benefícios esperados. Triage agrônômica imediata e de baixo custo (~R\$80 por nó de hardware), redução de perdas por diagnóstico tardio e democratização do acesso à informação técnica no campo.

4. Roadmap do Produto

O desenvolvimento foi organizado em **3 sprints de ~1 mês (4 semanas)**, além de uma fase futura registrada. A metodologia foi **Scrum adaptado** ao contexto individual, com revisão ao final de cada sprint e priorização por criticidade (Crítica → Baixa).



Figura 1 — Linha do tempo do produto (roadmap das 3 sprints).

Sprint	Tema	Duração	Tarefas	Status
1	Núcleo Inteligente (Backend + IA)	≈ 1 mês	18	Concluída
2	Borda Embarcada (Firmware + IoT)	≈ 1 mês	8	Concluída
3	Aplicativo e Integração (App + UX + Deploy)	≈ 1 mês	29	Concluída
—	Fase Futura (câmera OV5640, RPi3B+)	—	—	Registrada

Alterações de escopo (repriorizações justificadas): a câmera OV5640 foi mantida como prova de conceito (custo/prazo), com o app móvel assumindo o papel de produto principal; entre os experimentos de

IA, a augmentation sintética (Exp C) foi descartada por piorar o resultado, adotando-se fine-tuning com dados reais (Exp D) e Focal Loss (Exp E, modelo final).

5. Gestão e Tecnologias

5.1 Gestão do Projeto

- **Metodologia:** Scrum adaptado — sprints de ~1 mês com revisão e ajuste de backlog ao final de cada ciclo;
- **Sprints realizadas:** 3 (+ fase futura registrada);
- **Ferramenta de gestão e versionamento:** GitHub (github.com/Namem/extensao2) — histórico de commits (Conventional Commits) e backlog em Markdown/Excel;
- **Organização das tarefas:** backlog priorizado por criticidade, agrupado por épico e camada (Backend, IA, Firmware, App, Infra, QA);
- **Adaptação:** por ser projeto individual, os papéis de PO, Scrum Master e time foram acumulados pelo autor; a comunicação se deu via documentação viva no repositório.

5.2 Tecnologias Utilizadas

Tecnologia	Finalidade
Django REST Framework	Backend / API REST
PostgreSQL	Banco de dados (produção)
SimpleJWT	Autenticação por token JWT
Flutter / Dart	Aplicativo móvel e desktop
Drift (SQLite)	Persistência offline no app
flutter_map + OpenStreetMap	Mapa de ocorrências
geolocator	Captura de GPS
TensorFlow / Keras	Treinamento do modelo de IA
TensorFlow Lite / tflite_flutter	Inferência embarcada (celular e ESP32)
MobileNetV2 (INT8)	Arquitetura do classificador
rembg (U2-Net) / Pillow	Pré-processamento de imagens no treino
ESP32-S3 (PlatformIO / Arduino)	Firmware do nó IoT
DHT22 + sensor capacitivo	Sensores de temperatura, umidade do ar e do solo
MQTT (Mosquitto / HiveMQ Cloud)	Comunicação IoT (TLS / WebSocket)
Railway	Hospedagem do backend e do PostgreSQL
GitHub	Versionamento e gestão do backlog

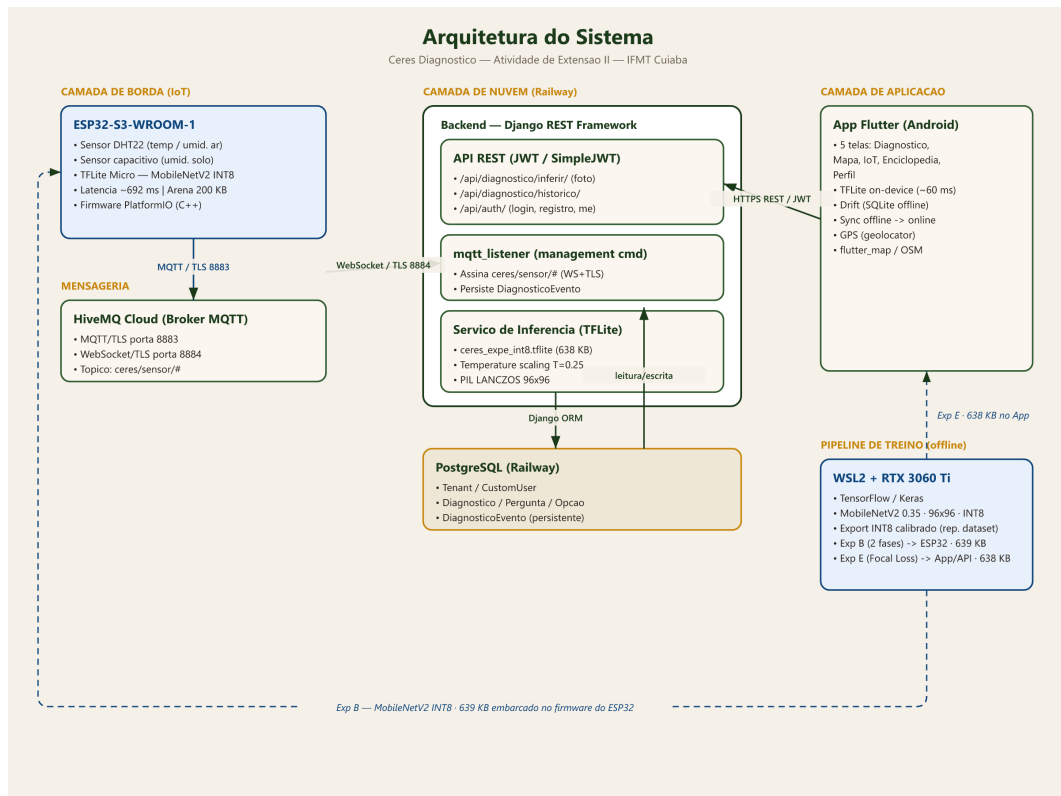


Figura 2 — Arquitetura do sistema em três camadas.

6. Descrição do Desenvolvimento

Esta seção detalha cada sprint com **todas as suas tarefas**, além de objetivo, dificuldades, decisões e entregas.

Sprint 1 — Núcleo Inteligente (Backend + IA) · ≈ 1 mês · 18 tarefas

Objetivo: construir a API Django, o motor de diagnóstico e o modelo de IA treinado e validado.

Épico A — Motor de Diagnóstico e API

Tarefa	Camada	Prioridade	Status
Mapeamento das 10 doenças do tomateiro (Embrapa)	Backend	Crítica	Concluído
Models Pergunta/Opcao/Diagnostico + migrations	Backend	Crítica	Concluído
Endpoints /iniciar/ e /responder/	Backend	Alta	Concluído
Autenticação JWT (SimpleJWT)	Backend	Alta	Concluído
Multi-tenant (Tenant + CustomUser)	Backend	Média	Concluído
PostgreSQL 18 (porta 5433)	Infra	Alta	Concluído
5 testes automatizados passando	QA	Alta	Concluído

Épico B — Dataset, Modelo de IA e MQTT

Tarefa	Camada	Prioridade	Status
PlantVillage (18.160 imgs) + augmentation → 88.949	IA	Crítica	Concluído
Exp A — Edge Impulse (FP32 92,5% / INT8 62%)	IA	Média	Concluído
Exp B — TF local (float 98,13% / INT8 95,76%, 639 KB)	IA	Crítica	Concluído
Exp C — Background augmentation sintética (20,24% campo)	IA	Baixa	Concluído
Exp D — Fine-tuning PlantDoc real (30,43% campo)	IA	Média	Concluído
Exp E — Focal Loss + aug agressiva (final, 638 KB)	IA	Crítica	Concluído
export_tflite.py — INT8 calibrado	IA	Alta	Concluído
Validação PlantDoc — gap lab-campo documentado	IA	Alta	Concluído
Backend MQTT (DiagnosticoEvento + mqtt_listener + /historico/)	Backend	Alta	Concluído
Mosquitto 2.1 instalado e testado	Infra	Média	Concluído
Matriz de confusão + acurácia por classe	QA	Média	Concluído

Dificuldades: TensorFlow sem suporte a Python 3.13 (uso de WSL2 + Python 3.12); detecção da GPU no WSL2; bug de pré-processamento INT8 na validação.

Decisões: treinar localmente (Exp B) para quantização INT8 calibrada; descartar a augmentation sintética (Exp C). **Entrega:** API funcional (5/5 testes) + modelo TFLite validado.

Sprint 2 — Borda Embarcada (Firmware + IoT) · ≈ 1 mês · 8 tarefas

Objetivo: rodar a IA e os sensores no hardware real ESP32-S3.

Épico A — Nó de sensores MQTT

Tarefa	Camada	Prioridade	Status
Projeto PlatformIO esp32_mqtt_sensor	Firmware	Alta	Concluído
WiFi + MQTT + reconexão automática	Firmware	Alta	Concluído
Publicação de JSON em ceres/sensor/	Firmware	Alta	Concluído
Pilha ESP32 → Mosquitto → Django → PostgreSQL	Integração	Alta	Concluído

Épico B — TFLite Micro no ESP32-S3

Tarefa	Camada	Prioridade	Status
esp32s3_ceres — TFLite Micro (arena PSRAM 200 KB)	Firmware	Crítica	Concluído

Tarefa	Camada	Prioridade	Status
gerar_arrays_c.py — modelo + 10 imgs como arrays C	IA	Alta	Concluído
Benchmark embarcado — 692 ms, 10/10 correto	Firmware	Crítica	Concluído
benchmark_esp32s3.md documentado	QA	Média	Concluído

Dificuldades: bibliotecas TFLite para ESP32 descontinuadas (solução: Chirale_TensorFlowLite); boot loop por flags de PSRAM; broker só escutando em localhost.

Decisões: remover a câmera OV5640 do escopo e usar imagens embutidas para validar a inferência.

Entrega: modelo rodando em hardware real com latência medida (692 ms).

Sprint 3 — Aplicativo e Integração (App + UX + Deploy) · ≈ 1 mês · 29 tarefas

Objetivo: entregar o aplicativo Flutter completo, publicado e resiliente.

Épico A — App base + inferência

Tarefa	Camada	Prioridade	Status
App Flutter (Diagnóstico + Histórico)	App	Crítica	Concluído
api_service — multipart POST + GET paginado	App	Alta	Concluído
View /inferir/ via subprocess TFLite	Backend	Crítica	Concluído
Validação end-to-end (galeria → resultado)	QA	Alta	Concluído
Experimento Edge vs Cloud	IA	Média	Concluído
Drift (SQLite) — persistência local	App	Alta	Concluído

Épico B — Design System e UI

Tarefa	Camada	Prioridade	Status
CeresTheme (paleta cerrado, Material 3)	App	Alta	Concluído
6 telas redesenhadas (Splash/Login/Diagnóstico/ IoT/Salvos/Enciclopédia)	App	Alta	Concluído
doencas_data.dart — 10 doenças centralizadas	App	Média	Concluído
flutter_svg + ícones thin-stroke	App	Baixa	Concluído
Tab bar + appbar fiéis ao mockup	App	Baixa	Concluído

Épico C — Deploy, nuvem e novas telas

Tarefa	Camada	Prioridade	Status
12 telas migradas + 4 novas (Alertas/Parceiro/Cadastro/Agrônomos)	App	Alta	Concluído

Tarefa	Camada	Prioridade	Status
Registro de usuário /register/	Backend	Alta	Concluído
Deploy Railway — ceres.up.railway.app	Infra	Crítica	Concluído
TFLite on-device Android (~60 ms)	App	Crítica	Concluído
MQTT Cloud HiveMQ (TLS 8883 / WS 8884)	IoT	Alta	Concluído

Épico D — UX: navegação, mapa e perfil

Tarefa	Camada	Prioridade	Status
Back button no CeresAppBar	App	Baixa	Concluído
Persistência de sessão + auto-refresh JWT	App	Média	Concluído
Banner offline (connectivity_plus)	App	Média	Concluído
latitude/longitude em DiagnosticoEvento	Backend	Média	Concluído
MapaScreen (flutter_map + OSM, marcadores)	App	Alta	Concluído
Captura de GPS no diagnóstico	App	Média	Concluído
GET /api/auth/me/ (estatísticas do usuário)	Backend	Média	Concluído
PerfilScreen (stats, exportar CSV, logout)	App	Média	Concluído

Épico E — Robustez e sincronização offline

Tarefa	Camada	Prioridade	Status
Railway PostgreSQL persistente entre deploys	Infra	Crítica	Concluído
Per-user diagnostics (FK usuario)	Backend	Alta	Concluído
Temperature scaling INT8 (T=0.25)	Backend/App	Alta	Concluído
Fila de sincronização offline → online (SyncService)	App	Alta	Concluído
Matriz de confusão INT8 (95,76%, 2.734 imgs)	QA	Média	Concluído

Dificuldades: confiança achatada do INT8 (temperature scaling T=0.25); race condition no GPS; Railway bloqueando portas MQTT (WebSocket + renome de env vars); SQLite efêmero (migração para PostgreSQL persistente).

Decisões: app como produto principal; design fiel a um mockup de referência. **Entrega:** aplicativo publicado + API em produção (ceres.up.railway.app).

7. Resultados Obtidos

Produto final e funcionalidades

- Diagnóstico de 10 categorias (9 doenças + saudável) por foto, cloud ou offline;
- Histórico de diagnósticos e leituras IoT em tempo real (temperatura, umidade do ar/solo);
- Mapa georreferenciado de ocorrências; enciclopédia das doenças; perfil com exportação CSV;
- Sincronização automática dos diagnósticos feitos offline ao voltar a conexão.

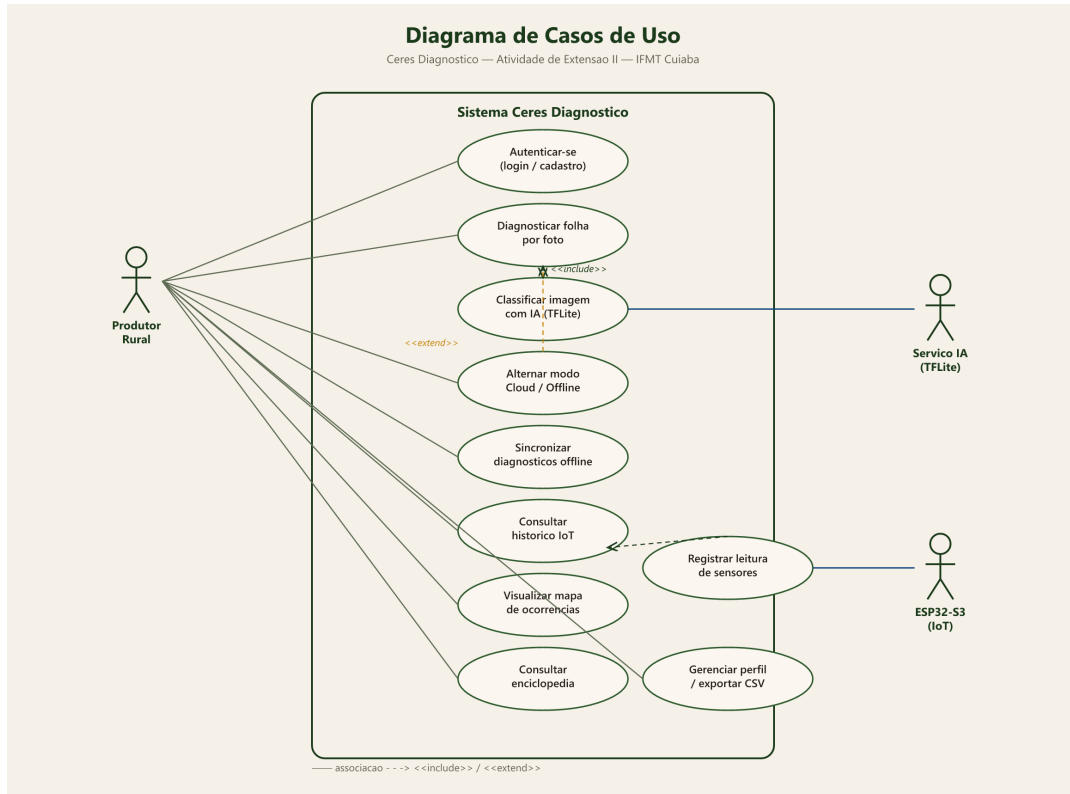
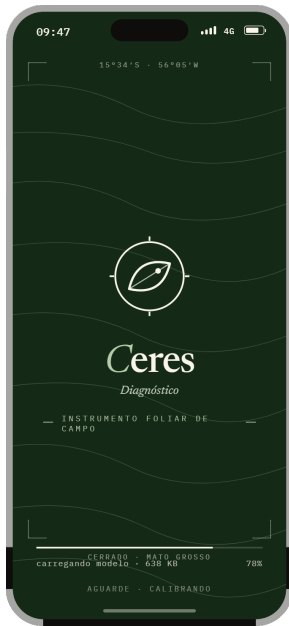


Figura 3 — Diagrama de casos de uso do sistema.

Principais telas do sistema

Capturas das telas do aplicativo (renderizadas a partir do protótipo de alta fidelidade):



Abertura



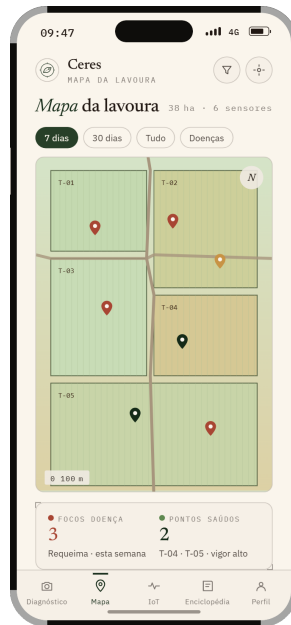
Login



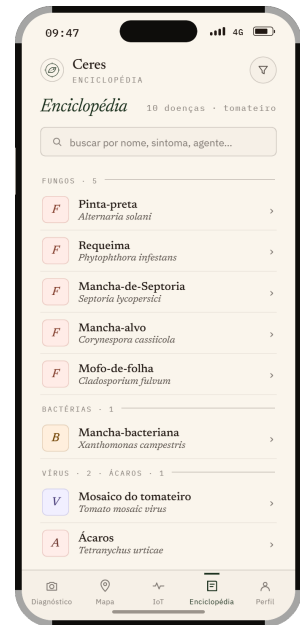
Diagnóstico (foto)



Histórico IoT



Mapa da lavoura



Enciclopédia

Métricas

Métrica	Valor
Acurácia (float, laboratório)	98,13%
Acurácia (INT8 embarcado, laboratório)	95,76%
Acurácia em campo real (documentada)	20–30%
Tamanho do modelo (INT8)	638 KB
Latência — ESP32-S3	692 ms
Latência — on-device (celular)	~60 ms
Latência — nuvem	~280–330 ms
Custo do nó de hardware	~R\$80

Testes e validações

- 5/5 testes automatizados no backend (motor de diagnóstico, MQTT, histórico);
- Validação end-to-end (galeria → inferência → resultado) no app;
- Matriz de confusão INT8 em 2.734 imagens (95,76%);
- Benchmark do ESP32-S3 (10/10 corretos) e experimento Edge vs Cloud.

Benefícios proporcionados

Triagem agrônômica imediata, de baixo custo e funcionando sem internet — colocando na mão do pequeno produtor uma ferramenta de apoio à decisão que antes dependia de especialista.

Limitações identificadas

- **Gap laboratório-campo:** o modelo, treinado com imagens de fundo controlado (PlantVillage), cai para 20–30% em fotos de campo real — fenômeno documentado na literatura (Mohanty 2016; Singh 2020). É a principal limitação e o foco dos próximos passos.
- **Câmera embarcada (OV5640):** não integrada ao ESP32-S3 na entrega (prova de conceito).
- **Validação com usuários reais:** ainda não realizada com produtores em campo — planejada como etapa futura.

Reflexão

- **Aprendizados:** transfer learning e quantização INT8 calibrada; comunicação IoT segura (MQTT/TLS); arquitetura offline-first; deploy em nuvem; a importância de documentar resultados negativos (o gap de campo não é fracasso — é achado).
- **Dificuldades enfrentadas:** compatibilidade de ambientes (GPU/WSL2), bibliotecas de TinyML descontinuadas, restrições de rede em PaaS.
- **Melhorias futuras:** dataset brasileiro de tomateiro, retreino com dados locais, câmera embarcada e validação com produtores.
- **Continuidade:** o projeto tem sequência natural como TinyML agrícola — “o gargalo é o dado, não o hardware”.

8. Anexos

8.1 Repositório GitHub

<https://github.com/Namem/extensao2> — código-fonte (backend, app, firmware), histórico de commits e README com instruções.

8.2 Slides das Sprint Reviews

Apresentação Sprint MVP e roteiro de apresentação (pasta slides/ do repositório).

8.3 Prints e fotos do projeto

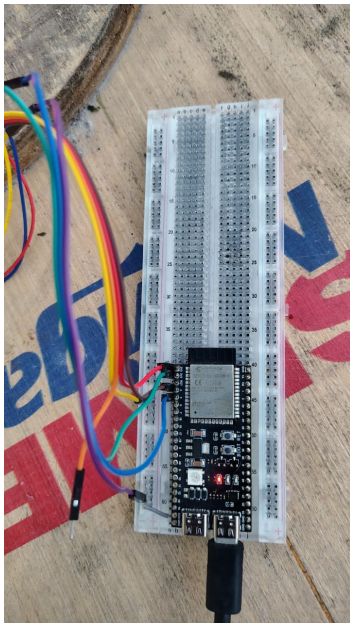
Hardware — nó IoT ESP32-S3 e testes



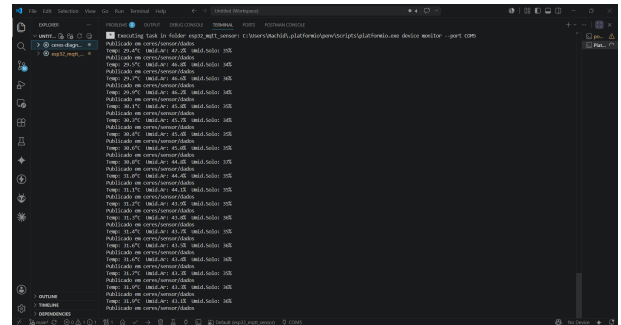
Nó completo: ESP32-S3 + DHT22 + sensor de solo no vaso



Sensor capacitivo de umidade do solo instalado

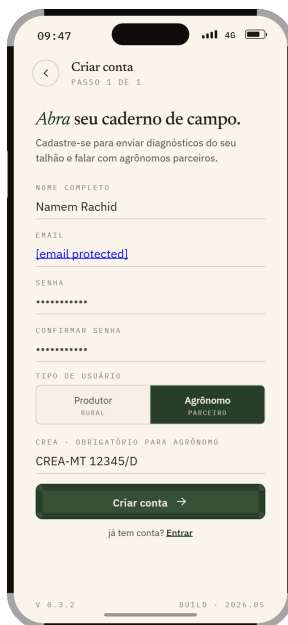


ESP32-S3 e ligações na protoboard



Monitor serial — leituras publicadas via MQTT

Aplicativo — demais telas



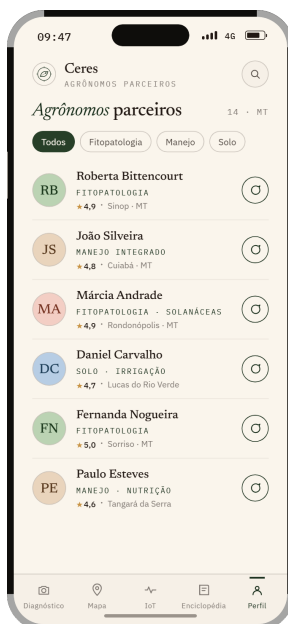
Cadastro



Salvos offline



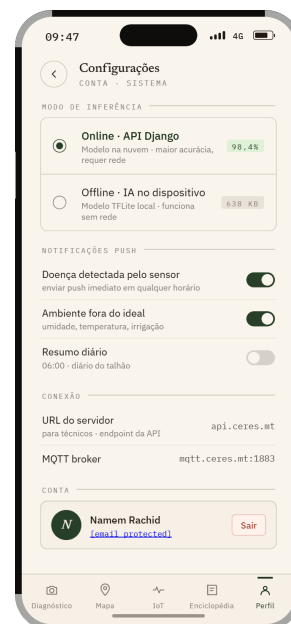
Central de alertas



Agrônomos parceiros



Seja parceiro



Configurações / Perfil

8.4 Documentação Técnica

A documentação técnica completa está reproduzida a seguir (diagramas, modelo de dados, documentação da API e manuais de instalação e do usuário).

8.4.1 Diagrama de Classes

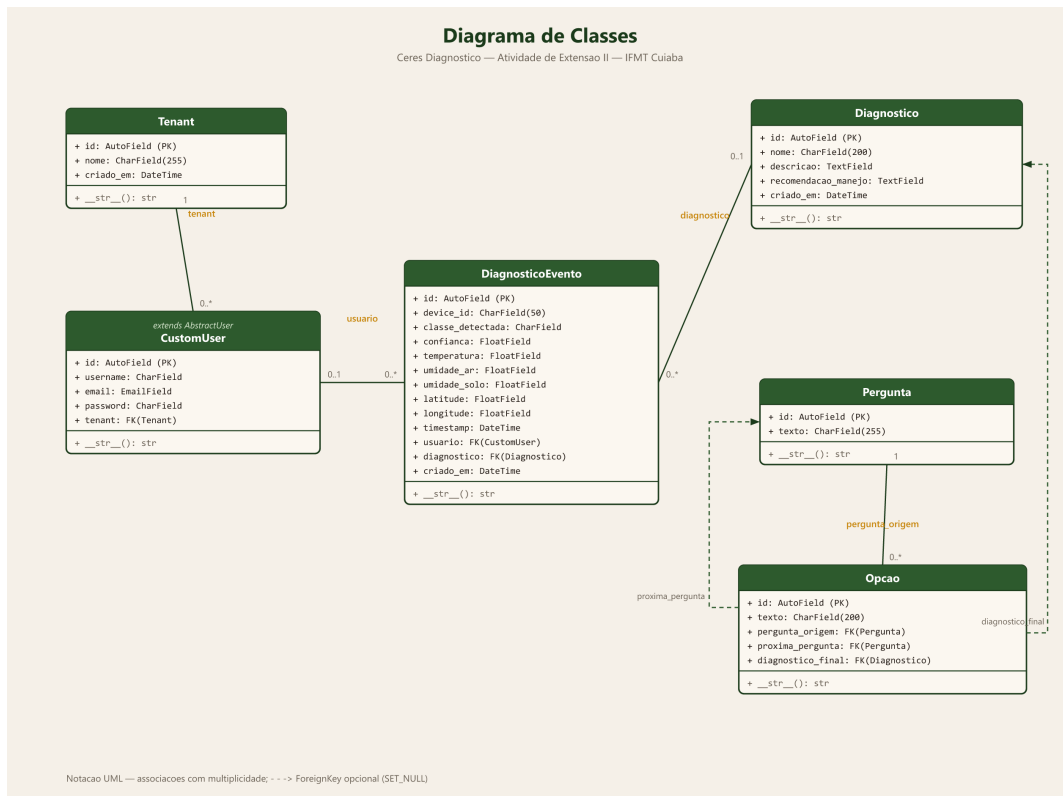


Figura 4 — Diagrama de classes (models do backend).

8.4.2 Modelo Entidade-Relacionamento (MER)

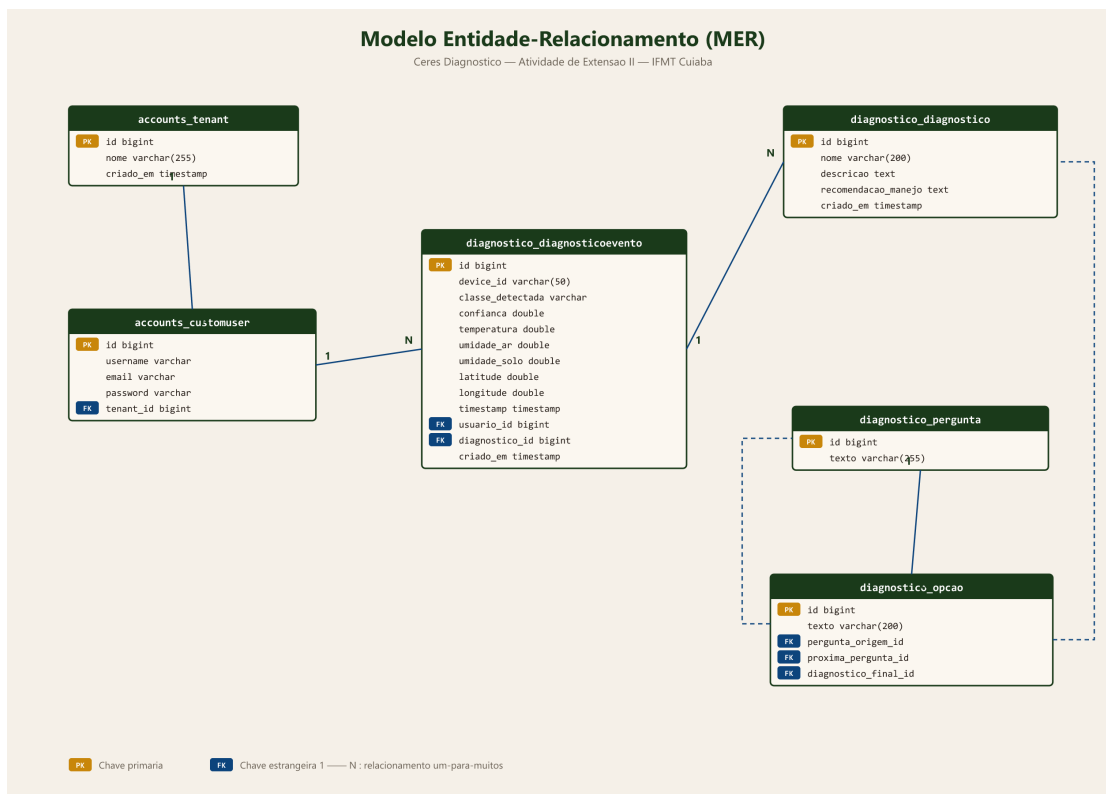


Figura 5 — Modelo Entidade-Relacionamento (esquema do banco).

8.4.3 Documentação da API REST

API construída em Django REST Framework. Base de produção <https://ceres.up.railway.app/api/> · base local <http://localhost:8080/api/>. Formato JSON (exceto /inferir/, multipart/form-data). Autenticação JWT (Bearer). A maioria dos endpoints é pública (o produtor usa o diagnóstico sem login); os dependentes do

usuário exigem o header Authorization: Bearer .

Método	Rota	Auth	Descrição
POST	/api/auth/token/	—	Login: retorna access + refresh (JWT)
POST	/api/auth/token/refresh/	—	Renova o access token
POST	/api/auth/register/	—	Cadastro de produtor/agrônomo
POST	/api/auth/reset-password/	—	Redefine senha (sem e-mail)
GET	/api/auth/me/	JWT	Perfil + estatísticas do usuário
GET	/api/diagnostico/iniciar/	—	Pergunta raiz da árvore de sintomas
POST	/api/diagnostico/responder/	—	Avança na árvore (próxima ou diagnóstico)
POST	/api/diagnostico/inferir/	—	Classifica a foto da folha (TFLite)
GET	/api/diagnostico/historico/	opcional	Histórico paginado (diagnósticos + IoT)
GET	/api/diagnostico/sensor/	—	Última leitura de sensor do ESP32

POST /api/diagnostico/inferir/ — recebe a imagem (campo imagem, multipart) e, opcionalmente, latitude/longitude; persiste o resultado como DiagnosticoEvento. Resposta 200:

```
{
  "classe": "D01_requeima",
  "class_index": 0,
  "confianca": 0.857,
  "latencia_ms": 279,
  "scores": { "D01_requeima": 0.857, "D09_mancha_bacteriana": 0.081, "...": ... }
}
```

Erros: 400 imagem ausente · 503 modelo indisponível · 504 timeout (>30s) · 500 falha. As 10 classes: D01_requeima, D02_septoriose, D03_pinta_preta, D03b_mancha_alvo, D05_mofo_foliar, D06_vira_cabeca, D06b_mosaico, D07_acaro_bronzeamento, D09_mancha_bacteriana, saudavel.

GET /iniciar/ e **POST /responder/** implementam o motor por árvore de sintomas. Resposta final de exemplo:

```
{ "tipo": "diagnostico",
  "dados": { "id": 3, "nome": "Requeima",
             "descricao": "...", "recomendacao_manejo": "..." } }
```

GET /historico/ — lista paginada (page_size 10, máx. 20). Autenticado: diagnósticos do próprio usuário + eventos IoT; anônimo: apenas eventos IoT. Cada item segue o modelo DiagnosticoEvento:

Campo	Tipo	Descrição
id	bigint (PK)	Identificador
device_id	varchar(50)	app_flutter, app_ ou ceres-esp32-01
classe_detectada	varchar	Classe da doença (nulo p/ leitura de sensor)
confianca	float	Probabilidade 0.0–1.0
temperatura / umidade_ar / umidade_solo	float	DHT22 e sensor capacitivo

Campo	Tipo	Descrição
latitude / longitude	float	GPS do celular no diagnóstico
timestamp / criado_em	datetime	Captura / recebimento no servidor
usuario	FK → CustomUser	Autor (nulo para evento MQTT do ESP32)
diagnostico	FK → Diagnostico	Diagnóstico associado (opcional)

Comunicação IoT (fora do REST): o ESP32-S3 publica via MQTT no broker HiveMQ Cloud (TLS 8883 / WebSocket 8884, tópico ceres/sensor/#); o comando Django `mqtt_listener` consome e persiste no PostgreSQL. Payload:

```
{"device_id":"ceres-esp32-01","temperatura":32.7,
  "umidade_ar":41.8,"umidade_solo":34}
```

8.4.4 Manual de Instalação

Guia para instalar e executar o sistema completo (Backend Django, App Flutter e Firmware ESP32-S3). Testado em Windows 11. Pré-requisitos: Git; Python 3.13 (Windows) / 3.12 (WSL2); PostgreSQL 18 porta 5433 (opcional — ou SQLite); Flutter SDK 3.44+; Visual Studio Build Tools (Desktop C++); PlatformIO 6.1+; Mosquitto 2.1+ (opcional).

1) Clonar:

```
git clone https://github.com/Namem/extensao2 ceres-diagnostico
cd ceres-diagnostico
```

2) Backend — Django REST API:

```
cd backend
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate --settings=ceres_core.settings_notebook
python manage.py test diagnostico
python manage.py runserver 0.0.0.0:8080 --settings=ceres_core.settings_notebook
```

A API sobe em `http://localhost:8080/api/`. Banco: SQLite (`settings_notebook`) ou PostgreSQL 18 (porta 5433). Copie `backend/.env.example` → `backend/.env` e ajuste (`SECRET_KEY`, banco, MQTT, `ALLOWED_HOSTS`) — nunca commitar o `.env`. Listener IoT: `python manage.py mqtt_listener --settings=ceres_core.settings_notebook`.

3) App — Flutter:

```
cd ..\app_ceres
flutter pub get
flutter run -d windows      # desktop
flutter build apk --release  # APK para celular
```

Ajuste o servidor em `lib/config.dart` (`baseUrl`): produção `https://ceres.up.railway.app` · local `http://localhost:8080` · emulador `http://10.0.2.2:8080` · celular na mesma WiFi `http://192.168.X.X:8080`. O APK sai em `build/app/outputs/flutter-apk/app-release.apk`.

4) Firmware — ESP32-S3: dois projetos em `firmware/`: `esp32_mqtt_sensor/` (sensores DHT22 + solo via MQTT) e `esp32s3_ceres/` (benchmark TFLite Micro). Copie `include/config.h.example` → `include/config.h` e ajuste WiFi/broker (`config.h` é ignorado pelo Git). Gravar:


```
cd firmware/esp32_mqtt_sensor
pio run --target upload --upload-port COM5
pio device monitor
```

Placa esp32-s3-devkitc-1, Flash 16MB, framework Arduino. Fazer no notebook (mesma rede WiFi do ESP32). **Deploy (Railway):** automático via git push na main; DJANGO_SETTINGS_MODULE=ceres_core.settings_railway (PostgreSQL via DATABASE_URL); URL <https://ceres.up.railway.app> · conta de teste test@test.com / test123. **Modelos TFLite:** ceres_expe_int8.tflite (Exp E, 638 KB — backend + app) e ceres_mobilenetv2_int8.tflite (Exp B, 639 KB — ESP32-S3).

8.4.5 Manual do Usuário

Guia de uso do aplicativo Ceres Diagnóstico — diagnóstico de doenças do tomateiro por foto, com IA embarcada no celular e monitoramento de sensores IoT. **Instalação:** copie o ceres_diagnostico.apk para o celular Android, permita “instalar de fonte desconhecida” e abra o app (há também versão web/desktop conectada à API em produção).

Primeiro acesso: o app abre no splash e vai ao Login. Opções: Entrar (e-mail + senha); Criar conta (Produtor ou Agrônomo — o agrônomo informa o CREA); Esqueci a senha (redefine pelo e-mail); Continuar sem conta (diagnóstico funciona; histórico na nuvem indisponível).

Telas principais — 5 abas:

- **Diagnóstico:** tire a foto (câmera ou galeria); o app mostra a doença (ou Saudável), a confiança (%), os scores e a recomendação de manejo (Embrapa); o resultado é salvo automaticamente. Modo Cloud (envia ao servidor, entra no mapa) ou Offline (IA no celular, ~60 ms, sem internet — nenhuma imagem sai do aparelho).
- **Mapa:** ocorrências georreferenciadas (OpenStreetMap), marcador por urgência; toque para ver doença, data, confiança e coordenadas.
- **IoT:** leituras do ESP32-S3 em tempo real — temperatura e umidade do ar (DHT22), umidade do solo (capacitivo), histórico e status MQTT.
- **Enciclopédia:** fichas das 10 categorias (sintomas, agente, ação) com busca.
- **Perfil:** nome, e-mail, estatísticas, modo de inferência, exportar CSV e sair.

As 10 categorias: D01 Requeima, D02 Septoriose, D03 Pinta-preta, D03b Mancha-alvo, D05 Mofo-foliar, D06 Vira-cabeça (TYLCV), D06b Mosaico, D07 Ácaro-do-bronzeamento, D09 Mancha-bacteriana e Saudável.

Uso offline e sincronização: sem internet, use o modo Offline; os diagnósticos ficam marcados como “não sincronizado” e, ao voltar a conexão, são enviados automaticamente para a nuvem. **Dicas:** fotografe a folha bem iluminada, preenchendo o quadro (fotos de campo com fundo natural reduzem a confiança — limitação conhecida). A confiança é uma estimativa: em caso de dúvida, consulte um agrônomo — o app é uma ferramenta de triagem.